

Real-Time Cybersecurity Threat Detection and Response System

Amanpreet Kaur, Lovnish Thakur, Shasank Suman, Nikhil

^{1,2,3}*Department of Computer Science and Engineering, Chandigarh University, Gharuan, Mohali, India*
amanpreet.e15674@cumail.in, 23BCS12489@cuchd.in, 23BCS12503@cuchd.in, 23BCS11134@cuchd.in

Abstract—Modern organizations generate large volumes of security telemetry from endpoints, networks, authentication services, and applications, yet conventional monitoring tools such as isolated intrusion detection systems and signature-based antivirus solutions struggle to correlate this information and identify sophisticated, behaviorally driven attacks. This results in alert fatigue, fragmented visibility, and delayed incident response within security operations centers (SOCs). This paper proposes a Real-Time Cybersecurity Threat Detection and Response System, a layered architecture designed to ingest and normalize multi-source security events, apply behavioral anomaly detection, and classify potential attacks using machine learning. The proposed system further integrates threat intelligence enrichment, a centralized SOC dashboard for analyst investigation, and controlled automated response playbooks governed by authorization and logging safeguards. The behavioral anomaly detection component is designed with a target precision of at least 90%, which is stated explicitly as a design objective rather than a measured result, pending validation on a suitable labeled security-event dataset. The paper presents the proposed system architecture, its constituent layers, and an evaluation methodology intended to guide future experimental validation, without presenting fabricated performance figures. The proposed approach is intended to improve centralized visibility, reduce manual investigation effort, and support faster, more consistent incident response in enterprise security operations.

Keywords—Cybersecurity, Threat Detection, Behavioral Anomaly Detection, Machine Learning, SIEM, Security Operations Center, Incident Response, Automated Response

I. INTRODUCTION

Cyberattacks against enterprise networks have grown steadily more sophisticated, spanning endpoint compromise, network intrusion, credential abuse, application-layer exploitation, and abuse of legitimate system utilities to evade detection. Attackers increasingly rely on techniques that blend into normal activity rather than triggering static, signature-based defenses, which makes isolated security tools such as standalone antivirus software or perimeter firewalls insufficient on their own. At the same time, the volume of security telemetry produced by modern IT environments, including authentication logs, network flow records, endpoint activity, and application logs, has grown far beyond what security analysts can manually review.

Conventional security monitoring approaches suffer from several well-documented limitations. Alert fatigue arises when analysts are presented with a high volume of low-fidelity alerts, many of which are false positives, making it difficult to prioritize genuine threats. Security data is frequently fragmented across disconnected tools, each with its own logging format, which hinders correlation of related

events. Static, rule- and signature-based detection struggles against novel or behaviorally subtle attacks that do not match a known pattern. These limitations, combined with slow manual investigation workflows and a lack of centralized visibility, contribute to delayed detection and response, extending the time attackers have to operate undetected within a network [4, 3].

These challenges motivate an integrated platform that combines SIEM-style multi-source collection, behavioral analytics, machine-learning-based classification, event correlation, threat-intelligence enrichment, centralized visualization for SOC analysts, and controlled automated response. Rather than treating detection and response as separate, manually bridged activities, such a platform aims to shorten the interval between the occurrence of suspicious activity and an appropriate defensive action, while keeping a human analyst positioned to review and, where necessary, override automated decisions [1, 7].

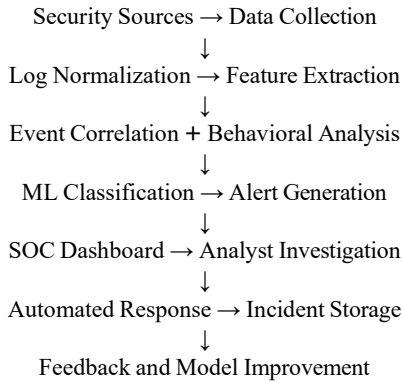
The contribution of this paper is a proposed system architecture, rather than a fully validated production deployment, comprising: (1) multi-source security event ingestion and normalization; (2) behavioral anomaly detection over user, host, and network activity; (3) machine-learning-based classification of detected activity into candidate attack categories; (4) a centralized SOC dashboard supporting analyst investigation; (5) automated response playbooks with authorization and audit safeguards; (6) incident correlation, prioritization, and record-keeping; and (7) an architecture intended to be extensible toward additional data sources and detection models. The novelty claimed here is architectural and integrative in nature: the individual techniques discussed (e.g., isolation-based anomaly detection, ensemble classification, threat-intelligence correlation) are drawn from established literature, and the contribution lies in combining them into a coherent, safeguarded detection-and-response pipeline suitable for SOC use.

II. PROPOSED SYSTEM

The proposed platform is a layered cybersecurity monitoring and response system that ingests security telemetry from multiple sources, analyzes it for anomalous or malicious behavior, and supports both analyst-driven and controlled automated response. All processed events, alerts, and response actions are retained in an auditable data store, and access to sensitive functions is restricted through authentication and role-based authorization.

A. System Overview

The system follows a staged processing flow:



Security sources such as authentication systems, network monitoring tools, and endpoint agents forward raw events to the collection stage. These events are parsed and normalized into a common schema, from which relevant features are extracted for both rule-based and machine-learning-based analysis. Behavioral analysis establishes a baseline of expected activity and flags statistically unusual behavior, while the classification component assigns a candidate attack category and confidence score to flagged activity. Enriched, correlated alerts are surfaced on the SOC dashboard for analyst review; depending on severity and policy, a bounded automated response playbook may also be triggered. Every alert, analyst action, and automated response is written to the incident data store, and periodically reviewed data may be used to retrain or recalibrate the detection models.

B. Key Features of the System

1) Multi-Source Security Integration. The platform is designed to accept security information from heterogeneous sources, including Linux system and authentication logs, Windows event logs where applicable, network monitoring and flow data, firewall logs, intrusion detection/prevention system (IDS/IPS) alerts, endpoint telemetry, application logs, existing SIEM exports, and external threat-intelligence feeds. Not every source listed here is assumed to be implemented in an initial prototype; the architecture is designed to be extended incrementally, and any specific source integration should be validated before being claimed as operational. Regardless of origin, ingested events are normalized into a common event representation (timestamp, source, entity, action, outcome, and contextual metadata) to make downstream correlation and analysis source-agnostic, an approach consistent with established SIEM data-source integration practice [8].

2) Behavioral Anomaly Detection. This component models the normal behavior of users, hosts, and network entities and flags statistically significant deviations, such as unusual login times or locations, repeated authentication failures, abnormal process execution, or atypical data-access patterns. Candidate unsupervised techniques discussed in the literature include Isolation Forest, One-Class SVM, Local Outlier Factor, density-based clustering, statistical control-chart methods, and autoencoder-based reconstruction error [6, 5]. For the proposed prototype, isolation-based modeling is selected as the primary candidate technique because it scales roughly linearly with sample size, requires no assumption of a partic-

ular data distribution, and produces an interpretable anomaly score without needing labeled attack data for initial baselining; this does not preclude evaluating alternative techniques during development.

3) ML-Based Attack Classification. Once an event or event cluster is flagged as anomalous or matches a correlation rule, a supervised classification stage assigns it to a candidate category such as brute-force activity, credential abuse, scanning/reconnaissance, malware-related indicators, denial-of-service activity, suspicious privilege escalation, or possible data-exfiltration indicators. The classification pipeline consists of feature extraction from normalized events, pre-processing (e.g., scaling and encoding), training on labeled historical data where available, validation, and confidence-scored inference. Ensemble tree-based methods such as Random Forest and gradient-boosted trees (e.g., XGBoost) are reasonable candidates given their established effectiveness on structured, tabular security-event features and their relative interpretability compared to deep neural architectures [3]; simpler baselines such as logistic regression are also useful for establishing a performance floor before adopting more complex models. Where feasible, predicted categories can additionally be mapped to a standardized adversary-behavior taxonomy such as MITRE ATT&CK, which provides a common vocabulary of tactics and techniques and can help analysts interpret classifier output in terms of an established attack lifecycle rather than an arbitrary label set [2].

4) Threat Intelligence Integration. Observed indicators, such as source IP addresses, domains, URLs, and file hashes, can be correlated against external threat-intelligence indicators of compromise to add context and confidence to an alert. This enrichment step does not itself constitute detection; rather, it adjusts the priority and interpretation of activity already flagged by rule-based or behavioral analysis. Any specific external feed integrated into an implementation should be a real, currently maintained source, and its coverage and update cadence should be documented rather than assumed.

5) Automated Response Playbooks. Where policy permits, the system can trigger bounded automated actions such as blocking a malicious IP address at the firewall, disabling or restricting a compromised account, isolating an affected endpoint from the network, terminating a malicious process, or escalating an incident to an analyst. Every automated action passes through a validation and severity-assessment step, an authorization/policy check, and is logged with the ability to review or roll back the action where technically feasible. Fully autonomous, irreversible, or offensive actions are explicitly out of scope; the design goal is to reduce analyst workload on well-understood, low-risk actions while keeping a human in the loop for ambiguous or high-impact decisions.

6) SOC Dashboard. A centralized dashboard presents active alerts with severity, affected assets, classified attack category, anomaly score, an event timeline, source information, threat-intelligence matches, response status, and historical trends. The dashboard is intended to reduce the time analysts spend switching between disconnected tools and to

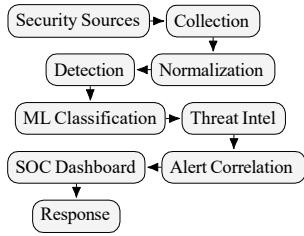


Figure 1: Overall system architecture and data flow.

give a consolidated view supporting triage and investigation.

7) Incident Management. Related alerts are grouped into incidents, prioritized by severity and asset criticality, and tracked through investigation, analyst annotation, response history, evidence retention, and closure, providing an auditable record for post-incident review and compliance reporting.

C. Benefits of the Proposed Approach

By centralizing multi-source telemetry and correlating it with behavioral and machine-learning-based analysis, the proposed approach is intended to reduce the manual effort required to identify related events across disparate tools, and to surface behaviorally anomalous activity that static rules alone would miss. Consolidating this information into a single SOC dashboard is intended to improve analyst situational awareness, while bounded automated response playbooks are intended to shorten the time between detection and containment for well-understood incident types. The layered architecture is designed to be extensible as new data sources or detection models become available. These are design objectives rather than guarantees: no detection system can eliminate false positives, false negatives, or novel attack techniques entirely, and the effectiveness of the proposed approach depends on the quality of the underlying data, the representativeness of any training data used, and the operational tuning performed by the security team.

III. SYSTEM ARCHITECTURE

The proposed system is organized into eight layers: (A) Data Collection, (B) Data Processing and Normalization, (C) Detection and Analytics, (D) Machine Learning, (E) Threat Intelligence and Correlation, (F) Response and Orchestration, (G) SOC Presentation, and (H) Data Storage. Figure 1 summarizes the overall architecture, and Figure 2 details the event-level detection and response workflow.

A. Data Collection Layer

This layer ingests raw telemetry from the sources described in Section II-B(1) using lightweight agents, log-forwarding protocols (e.g., syslog), or vendor APIs, depending on the source. Its responsibility is limited to reliable, timestamped collection; interpretation of the data is deferred to later layers.

B. Data Processing and Normalization Layer

Collected events are parsed, timestamp-aligned, and mapped into a common schema. Field extraction, deduplication of repeated log entries, basic enrichment (e.g., resolving an asset identifier), and validation of malformed records occur here. Normalization is essential because it allows the detection layer to correlate events originating from structurally different sources (for example, a firewall log and an authentication log) without source-specific logic at every downstream stage.

C. Detection and Analytics Layer

This layer applies rule-based and threshold detection (e.g., a fixed number of failed logins within a time window) alongside correlation logic that links related events across sources. Detected conditions are scored for severity. A representative, non-exhaustive conceptual formulation for an alert score is:

$$S = f(\sigma, \phi, c, a, \tau) \quad (1)$$

where σ is base event severity, ϕ is event frequency within a defined window, c is the confidence of the underlying detection (rule match or model output), a is the criticality of the affected asset, and τ is any threat-intelligence context available for the event. The function f is a weighted combination whose exact form (e.g., a weighted sum or a bounded aggregation) should be tuned against operational data and is not specified as fixed a priori.

D. Machine Learning Layer

This layer performs both anomaly scoring and supervised classification, as introduced in Section II-B(2)–(3). For anomaly detection, an anomaly score $a(x)$ is computed for a feature vector x representing an entity’s recent behavior, with higher scores indicating greater deviation from the learned baseline; for isolation-based methods, this corresponds conceptually to the average path length required to isolate x in an ensemble of random partitioning trees, with shorter paths indicating higher anomaly likelihood [5]. For classification, the predicted category is

$$y^* = f(X) \quad (2)$$

where X is the vector of extracted security-event features and f is the trained classification model, producing a predicted attack category y^* together with a confidence score. Model outputs, along with analyst feedback (e.g., confirming or dismissing an alert), are logged to support periodic retraining as the security environment evolves.

E. Threat Intelligence and Correlation Layer

Observed indicators are looked up against threat-intelligence sources, and any match is used to enrich the alert with context (e.g., known malicious infrastructure) and to adjust its confidence and priority:

$$\begin{aligned} \text{Observed Indicator} &\rightarrow \text{Intelligence Lookup} \rightarrow \text{Context/Confidence} \\ &\rightarrow \text{Alert Enrichment} \end{aligned}$$

This layer does not independently generate alerts; it augments alerts already produced by the detection and analytics layer.



Figure 2: Detection and response workflow at the event level.

F. Response and Orchestration Layer

Automated response follows a controlled pipeline:

Detection → Validation → Severity Assessment → Playbook
 Selection → Authorization Check → Response Action →
 Verification → Logging → Incident Update

Safeguards are necessary at each stage because an automated response acting on a false positive can itself cause operational disruption (e.g., blocking a legitimate user or isolating a production host). The authorization check enforces policy on which playbooks may run without human approval, and the verification step confirms that the intended action was applied correctly before the incident record is updated.

G. SOC Presentation Layer

This layer implements the dashboard described in Section II-B(6), including alert and severity visualization, an incident timeline, filtering by asset or category, and response-status tracking, structured to support the day-to-day triage workflow of SOC analysts.

H. Data Storage Layer

Normalized events, alerts, incidents, threat indicators, model predictions and scores, and response-action logs are persisted in a data store designed for indexed retrieval, horizontal scalability as event volume grows, configurable retention periods, and role-based access control. Integrity protections (e.g., append-only audit logging for response actions) help ensure that incident records remain trustworthy for post-incident review.

I. Security and Privacy Considerations

Because the platform aggregates sensitive operational and identity data, it requires strong authentication and role-based access control (RBAC) so that analysts and automated components only access data and actions appropriate to their role. Communication between layers should use encrypted channels, and credentials used for source integrations should be stored using a dedicated secrets manager rather than in configuration files. All administrative actions and automated response actions should be captured in an integrity-protected audit log. Least-privilege principles should govern both human accounts and any service accounts used by collection

Table 1: Representative Security Data Sources

Source	Example Events	Processing	Security Purpose
Authentication logs	Login success/failure, MFA events	Parse, normalize, rate-count	Credential-abuse and brute-force detection
Network monitoring	Flow records, connection metadata	Normalize, aggregate	Scanning and lateral-movement detection
Firewall/IDS-IPS	Blocked/allowed connections, signature hits	Normalize, correlate	Perimeter and known-signature detection
Endpoint telemetry	Process creation, file/registry activity	Normalize, feature-extract	Malware and privilege-escalation indicators
Application logs	API calls, error/audit events	Parse, normalize	Application-layer abuse detection

Table 2: Detection and Machine-Learning Components

Component	Technique	Input	Output
Behavioral anomaly detection	Isolation-based scoring	Entity behavior features	Anomaly score
Attack classification	Ensemble tree-based model	Normalized event features	Category + confidence
Correlation engine	Rule-based aggregation	Multi-source events	Correlated alert
Threat-intel enrichment	Indicator lookup	Observed IOCs	Context/confidence adjustment

agents or response connectors, and log data containing personally identifiable information should be handled in accordance with applicable data-retention and privacy requirements.

IV. RESULT AND DISCUSSION

Because no dataset-based experiment or production deployment has been carried out for this paper, the following subsections describe an evaluation methodology intended to guide future validation of the proposed architecture, rather than reporting measured performance. No accuracy, precision, recall, detection-rate, or response-time figures are claimed as achieved.

A. Evaluation Methodology

A rigorous evaluation of the proposed system would require a labeled security-event dataset, or a controlled test environ-

ment with known injected attack scenarios, against which the following metrics could be computed: precision, recall, F1-score, overall accuracy, false-positive rate, false-negative rate, detection latency, classification performance (via a confusion matrix), and end-to-end response execution time. Publicly available intrusion-detection datasets, alongside a controlled lab environment, are reasonable starting points for such an evaluation, provided their limitations (e.g., dataset age or lack of representativeness of current attack techniques) are accounted for in the interpretation of results.

B. Behavioral Anomaly Detection Evaluation

The stated design objective for the behavioral anomaly detection component is a *target* precision of at least 90%. This figure is a design target, not a demonstrated result, and its validity would need to be established by running the isolation-based (or comparable) detector against a labeled dataset containing both benign and confirmed-malicious behavioral sequences, then computing precision at an operationally chosen anomaly-score threshold. Because precision and recall trade off against one another as the threshold changes, evaluation should report precision at several operating points rather than a single figure.

C. ML Attack Classification Evaluation

Evaluating the classification component would require a held-out labeled test set spanning the candidate attack categories described in Section II-B(3), a computed confusion matrix to identify which categories are most frequently confused, and per-class precision/recall in addition to an overall accuracy figure, since class imbalance (e.g., far more benign events than confirmed exfiltration events) can make an aggregate accuracy figure misleading on its own.

D. System-Level Evaluation

Beyond detection accuracy, a system-level evaluation should examine sustained event-ingestion throughput, alert-processing latency from event arrival to alert generation, dashboard responsiveness under realistic alert volumes, correctness and latency of automated response orchestration, and horizontal scalability of the storage and processing layers as data volume grows. None of these have been measured for this paper; they are proposed as future evaluation criteria.

E. Discussion

The proposed architecture is expected to improve centralized visibility and reduce the manual correlation burden on SOC analysts by consolidating multi-source telemetry and surfacing behaviorally anomalous activity that purely signature-based tools would miss. The target evaluation criterion for anomaly-detection precision ($\geq 90\%$) reflects a reasonable design goal drawn from precedents in the anomaly-detection literature [6], but experimental validation on representative, current attack data is required to establish whether this target is achievable in a given deployment without an unacceptable false-positive burden. Likely limitations include sensitivity of behavioral baselines to legitimate changes in

user or system behavior (which can produce transient false positives), the need for continual retraining as both the environment and attacker techniques evolve, and the residual risk that a sufficiently novel attack technique evades both the anomaly-detection and classification components. Future testing should determine appropriate anomaly-score thresholds, evaluate the classifier's performance on evolving attack categories, and quantify the operational impact of the automated-response safeguards under realistic incident volumes.

V. CONCLUSION

This paper presented a proposed architecture for a Real-Time Cybersecurity Threat Detection and Response System that integrates multi-source security telemetry, behavioral anomaly detection, machine-learning-based attack classification, threat-intelligence enrichment, a centralized SOC dashboard, and safeguarded automated response playbooks. The system is organized into eight cooperating layers spanning collection, normalization, detection, machine learning, threat-intelligence correlation, response orchestration, presentation, and storage, with authentication, role-based access control, and audit logging applied throughout. The behavioral anomaly detection component is designed around a target precision of at least 90%, explicitly framed as a design objective pending experimental validation rather than an achieved result. The expected contribution of this work is architectural: a coherent integration of established detection and response techniques into a single, safeguarded pipeline intended to reduce alert fatigue and shorten the interval between detection and containment in SOC operations. The main limitation of this paper is the absence of experimental validation on a labeled dataset or in a live deployment; consequently, all effectiveness claims should be read as design targets. Realistic future work includes evaluating the architecture against public and internally curated labeled datasets, exploring deep-learning and federated-learning approaches for the anomaly-detection and classification components, improving threat-intelligence coverage and freshness, extending the response layer toward more advanced SOAR-style orchestration with explainable model outputs, assessing robustness against adversarial evasion, and conducting a controlled, enterprise-scale evaluation once the necessary infrastructure and data-sharing agreements are in place [9].

REFERENCES

- [1] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, "Computer Security Incident Handling Guide," NIST Special Publication 800-61, Revision 2, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2012.
- [2] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and Philosophy," Technical Report, The MITRE Corporation, Bedford, MA, USA, 2020.
- [3] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection,"

IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153–1176, 2016.

- [4] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
- [5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [6] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, Article 15, pp. 1–58, 2009.
- [7] M. Vielberth, F. Böhm, I. Fichtinger, and G. Pernul, “Security operations center: A systematic study and open challenges,” *IEEE Access*, vol. 8, pp. 227756–227779, 2020.
- [8] G. González-Granadillo, S. González-Zarzosa, and R. Díaz, “Security information and event management (SIEM): Analysis, trends, and usage in critical infrastructures,” *Sensors*, vol. 21, no. 14, Article 4759, 2021.
- [9] J. Kinyua and L. Awuah, “AI/ML in security orchestration, automation and response: Future research directions,” *Intelligent Automation & Soft Computing*, vol. 28, no. 2, pp. 527–545, 2021.